

МЕТОДИЧЕСКИЕ УКАЗАНИЯ

УП.03. Учебная практика

Этап 2. Проведение инсталляции ПО компьютерных систем

Параметр	Значение
Практика	УП.03. Сопровождение и обслуживание программного обеспечения компьютерных систем
Этап	2. Инсталляция, запуск и подготовка технического окружения
Основной результат	Рабочие команды запуска, Docker/Compose, Makefile, BAT-файлы, линтеры, форматтеры и доказательства запуска
Формат сдачи	Ссылка на репозиторий + короткий отчёт + скриншоты

Что должно получиться после этапа

После выполнения этапа проект должен запускаться не только “у меня в IDE”, а через понятные команды. Преподаватель или другой студент должен открыть репозиторий, выполнить команды и получить рабочую программу.

Что должно быть	Зачем это нужно	Минимальный результат
BAT-скрипты	Для запуска на Windows без ручного поиска команд	scripts/setup.bat, run.bat, check.bat, format.bat
Makefile	Единая карта команд проекта	make setup, make run, make check, make format
Dockerfile	Запуск проекта в изолированном окружении	Проект собирается в контейнер
docker-compose.yml	Запуск нескольких частей проекта одной командой	backend/frontend/db поднимаются через docker compose up --build
Линтер	Проверяет стиль и типовые ошибки	ruff/eslint/ktlint/dotnet format/gradle lint или аналог
Форматтер	Приводит код к единому виду	black/prettier/dotnet format/spotless или аналог
.env.example	Показывает настройки без реальных секретов	Есть пример переменных окружения
.dockerignore	Не кладёт мусор в Docker-образ	Исключены .git, node_modules, .venv, _pycache и т.д.

Структура сдачи

Сдаётся не папка с бесконечными текстами, а репозиторий, где появились файлы для запуска и проверки проекта.

```

УП03_Этап2_Иванов_Иван_Группа/
|-- repo_link.txt
|-- docs/
|   |-- 01_Короткий_отчет_по_этапу_2.docx
|-- screenshots/
|   |-- 01_clean_clone.png
|   |-- 02_setup_bat_success.png

```

```
| |-- 03_make_help_or_make_check.png
| |-- 04_docker_build_success.png
| |-- 05_docker_compose_up.png
| |-- 06_app_opened_in_browser.png
| |-- 07_linter_success.png
| |-- 08_formatter_success.png
| |-- 09_git_commit.png
|-- project_files_in_repository/
|   |-- Dockerfile
|   |-- docker-compose.yml
|   |-- .dockerignore
|   |-- Makefile
|   |-- .env.example
|   |-- .editorconfig
|   |-- scripts/
|   |   |-- setup.bat
|   |   |-- run.bat
|   |   |-- check.bat
|   |   |-- format.bat
|   |   |-- docker-up.bat
|   |   |-- docker-down.bat
|   |   |-- logs.bat
|   |-- README.md / INSTALL.md
```

Порядок выполнения

Шаг	Что сделать	Что приложить
1	Скачать проект в новую папку и проверить, что он запускается не из старой рабочей директории.	Скриншот 01_clean_clone.png
2	Добавить или исправить .env.example и .editorconfig.	Скриншот файлов в корне проекта
3	Создать scripts/setup.bat, run.bat, check.bat, format.bat.	Скриншоты успешного запуска BAT-файлов
4	Настроить линтер и форматтер под свой стек.	Скриншот успешной проверки check.bat
5	Добавить Dockerfile и .dockerignore.	Скриншот docker build
6	Добавить docker-compose.yml, если в проекте больше одной части или есть БД.	Скриншот docker compose up --build
7	Добавить Makefile с командами setup/run/check/format/docker-up/docker-down/logs.	Скриншот make check или make docker-up
8	Обновить README.md или INSTALL.md короткими командами запуска.	Скриншот фрагмента README
9	Сделать commit и push в репозиторий.	Скриншот commit/push

Обязательные команды проекта

У проекта должны быть короткие команды. Не допускается ситуация, когда студент говорит: “сейчас я вспомню, как его запускать”.

Действие	BAT-команда	Makefile-команда	Docker-команда
Установка зависимостей	scripts/setup.bat	make setup	docker compose build
Запуск проекта	scripts/run.bat	make run	docker compose up --build
Проверка качества	scripts/check.bat	make check	docker compose exec app ...
Форматирование	scripts/format.bat	make format	docker compose exec app ...
Просмотр логов	scripts/logs.bat	make logs	docker compose logs -f
Остановка контейнеров	scripts/docker-down.bat	make docker-down	docker compose down

ВАТ-файлы: обязательный минимум

ВАТ-файлы обязательны, потому что большинство студентов запускает проект на Windows. Каждый ВАТ должен начинаться с перехода в корень проекта.

Общий шаблон для любого ВАТ-файла

```
@echo off
chcp 65001 > nul
cd /d "%~dp0\.."

echo =====
echo  Название действия
echo  =====

REM дальше идут команды проекта
```

```
# scripts/docker-up.bat
@echo off
chcp 65001 > nul
cd /d "%~dp0\.."

docker compose up --build
```

```
# scripts/docker-down.bat
@echo off
chcp 65001 > nul
cd /d "%~dp0\.."

docker compose down
```

```
# scripts/logs.bat
@echo off
chcp 65001 > nul
cd /d "%~dp0\.."

docker compose logs -f
```

Makefile: единая карта команд проекта

Makefile нужен не вместо ВАТ-файлов, а вместе с ними. ВАТ удобен для Windows, Makefile показывает профессиональную структуру команд проекта.

Базовый Makefile

```
.PHONY: setup run check format docker-build docker-up docker-down logs clean

setup:
    @echo "Install dependencies"
    # сюда вставить команду установки зависимостей

run:
    @echo "Run project locally"
    # сюда вставить команду локального запуска

check:
```

```

@echo "Run checks"
# сюда вставить линтеры, тесты или сборку

format:
  @echo "Format code"
  # сюда вставить форматтер

docker-build:
  docker compose build

docker-up:
  docker compose up --build

docker-down:
  docker compose down

logs:
  docker compose logs -f

clean:
  # удалить временные файлы, если это безопасно для проекта

```

Docker: когда обязателен

Docker на этом этапе обязателен, если проект содержит backend, базу данных, несколько сервисов, нестандартные зависимости или если проект сложно запускать вручную. Для совсем простого консольного проекта допускается Dockerfile без docker-compose.yml, но причина должна быть указана в коротком отчёте.

Тип проекта	Что нужно сделать
Простой Python/Node/Java проект без БД	Dockerfile + .dockerignore. docker-compose.yml желательно, но можно без него.
Backend + база данных	Dockerfile + docker-compose.yml с сервисами app и db.
Frontend + backend	Dockerfile для каждой части или общий compose с frontend и backend.
Frontend + backend + БД	docker-compose.yml с frontend, backend, db, volumes и env file.
Android / Desktop	Docker может быть заменён сборкой через Gradle/.NET, но Makefile и BAT всё равно нужны.

Dockerfile: примеры

Python / FastAPI: Dockerfile

```

FROM python:3.12-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

```

Node.js / React / Vue: Dockerfile

```
FROM node:20-alpine

WORKDIR /app
COPY package*.json ./
RUN npm install

COPY . .
EXPOSE 5173

CMD ["npm", "run", "dev", "--", "--host", "0.0.0.0"]
```

Java / Spring: пример Dockerfile

```
FROM eclipse-temurin:21-jdk

WORKDIR /app
COPY . .
RUN ./mvnw -q -DskipTests package

EXPOSE 8080
CMD ["java", "-jar", "target/app.jar"]
```

docker-compose.yml: пример

Compose нужен, когда проект состоит из нескольких частей. Например: backend, frontend и база данных.

```
services:
  backend:
    build: ./backend
    env_file:
      - .env
    ports:
      - "8000:8000"
    depends_on:
      - db
    volumes:
      - ./backend:/app

  frontend:
    build: ./frontend
    ports:
      - "5173:5173"
    depends_on:
      - backend
    volumes:
      - ./frontend:/app

  db:
    image: postgres:16
    environment:
      POSTGRES_DB: app_db
      POSTGRES_USER: app_user
      POSTGRES_PASSWORD: app_password
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

.dockerignore: обязательный файл

Без .dockerignore в Docker-образ попадут лишние файлы: виртуальные окружения, кэш, node_modules, git-история, временные файлы.

```
.git
.gitignore
.venv
venv
__pycache__
*.pyc
node_modules
dist
build
.idea
.vscode
.env
*.log
.DS_Store
screenshots
docs
```

.env.example и режимы запуска

Настоящий .env не загружается в репозиторий. В репозитории должен быть только .env.example с примерными значениями.

```
APP_ENV=development
APP_DEBUG=true
APP_HOST=0.0.0.0
APP_PORT=8000

DATABASE_URL=postgresql://app_user:app_password@db:5432/app_db
SECRET_KEY=change_me_in_real_env

SMTP_HOST=localhost
SMTP_PORT=1025
```

Линтеры и форматтеры: что выбрать

Студент выбирает инструменты под свой стек. Нужно не перечислить инструменты в отчёте, а реально настроить команды check и format.

Стек	Линтер / проверка	Форматтер	Команды
Python	ruff check . + pytest	black . или ruff format .	scripts/check.bat, scripts/format.bat, make check, make format
Node.js / Frontend	eslint .	prettier --write .	npm run lint, npm run format
Java / Spring	mvn test, checkstyle/spotless	spotless:apply	mvn clean test
C# / .NET	dotnet build, dotnet test	dotnet format	dotnet format --verify-no-changes
Android	gradlew lint, gradlew test	ktlint/spotless	gradlew assembleDebug

Пример настройки Python

```
# pyproject.toml для Python-проекта
[tool.black]
line-length = 88
```

```
[tool.ruff]
line-length = 88

[tool.pytest.ini_options]
testpaths = ["tests"]
```

Пример настройки Node.js

```
// package.json для Node.js / Frontend
{
  "scripts": {
    "dev": "vite --host 0.0.0.0",
    "lint": "eslint .",
    "format": "prettier --write .",
    "check": "npm run lint"
  },
  "devDependencies": {
    "eslint": "latest",
    "prettier": "latest"
  }
}
```

Короткий отчёт

Отчёт нужен только для фиксации результата. Основная работа должна быть в репозитории.

Раздел отчёта	Что написать
1. Ссылка на репозиторий	Ссылка на GitHub/GitLab и ветка, где лежит работа.
2. Как запустить проект локально	3-5 команд: setup, run, адрес приложения.
3. Как запустить через Docker	docker compose up --build, адрес приложения, как остановить.
4. Какие инструменты качества настроены	Линтер, форматтер, тесты/сборка, команды запуска.
5. Какие проблемы были исправлены	1-3 конкретные проблемы: зависимость, порт, .env, Dockerfile, lint-ошибка.
6. Скриншоты	Список скриншотов с кратким пояснением.

Обязательные скриншоты

Файл	Что должно быть видно
01_clean_clone.png	Проект скачан/клонирован в новую папку.
02_setup_bat_success.png	Успешное выполнение scripts/setup.bat.
03_make_help_or_make_check.png	Выполнение make check или вывод Makefile-команд.
04_docker_build_success.png	Успешная сборка Docker-образа.
05_docker_compose_up.png	Запущенные сервисы через docker compose up --build.
06_app_opened_in_browser.png	Открытая программа в браузере/окне/терминале.
07_linter_success.png	Линтер выполнен или показаны найденные и исправленные ошибки.
08_formatter_success.png	Форматтер выполнен.
09_git_commit.png	Коммит с добавленными файлами и push в репозиторий.

Критерии приемки этапа

Критерий	Принимается	Не принимается
Запуск	Проект запускается через ВАТ и/или Docker.	Проект запускается только вручную через IDE.
Docker	Есть Dockerfile, .dockerignore, compose при необходимости.	Docker отсутствует без объяснения.
Makefile	Есть команды setup/run/check/format/docker-up/docker-down/logs.	Makefile пустой или содержит команды, которые не работают.
Линтер/форматтер	Настроены и запускаются одной командой.	Только написано название инструмента, но команды нет.
Секреты	Есть .env.example, настоящий .env не загружен.	В репозитории лежат реальные пароли/токены.
Скриншоты	Показывают реальные команды и результат.	Скриншоты не доказывают запуск.